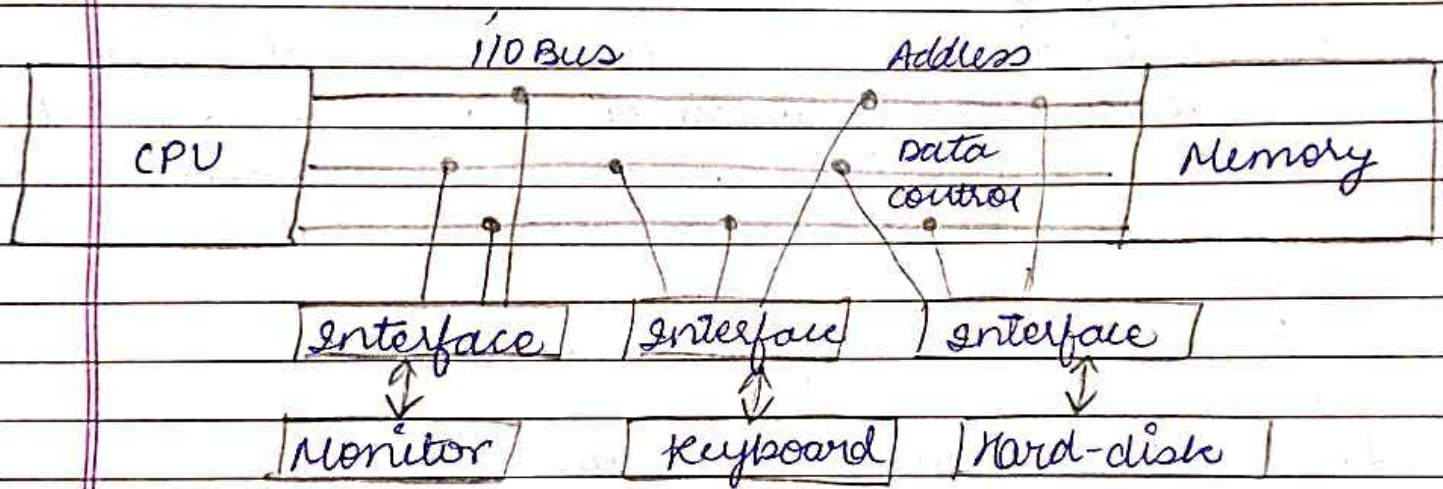


I/O Devices Interface:

Input-output interface is used as a method which helps in transferring the information b/w the internal storage devices i.e. memory and the external peripheral devices.

A peripheral device is that which provide input and output for the computer ~~etc.~~

like a keyboard and a mouse provide an input, while a printer and monitor provides output.



In micro computer based system, the only purpose of peripheral devices is just to provide special communication links for the interfacing them with the CPU. To overcome the difference b/w CPU and peripheral devices, ~~there~~ there is a special unit for communication links.

Difference B/w Peripheral devices and CPU.

Peripheral devices

CPU

- The nature is electro-magnetic.
 - Synchronization mechanism needed because the data rate is fast. transfer rate is slow.
 - data codes and formats are very different.
 - Controlling of peripheral devices needed as it does not disturbs the operation of other devices connected to CPU.
- The nature is Electronic.
 - The data transfer rate is fast.
 - It shares the data codes and formats with main memory.
 - No control is required as CPU works itself and properly.

Functions of input/output interface:

1. Used to synchronize the operating speed of CPU with respect to input-output devices.
2. It selects the input/output device, which is appropriate for the interpretation of the input/output signal.
3. It is capable of providing signals like control and timing signals.

- Date _____
Page _____
4. In this, data buffering can be possible through data Bus.
 5. There are various error detectors.
 6. It converts serial data into parallel data & vice-versa.
 7. It also converts digital data into analog data and vice-versa.

Keyboard and Display interface

A process of connecting a keyboard to a computer or a microcontroller, allowing a user to input data or commands using the keys on the keyboard.

It involves the electrical & logical connection b/w the keyboard and the device it is being connected to.

Bus Control:

It manages the communication b/w the computer's CPU and its other components. It specializes in transferring control signals that coordinate and regulate the hardware's actions. The signals ensures operations are carried out in the correct sequence and at the right time.

How the Bus control works?

In a computer's architecture, the bus system consists of the data, address and control buses.

- The data bus $\rightarrow$ transfers data.
 - The address bus $\rightarrow$ carries memory addresses where the data is stored/retrieved.
 - Control Bus conveys command signals.
- These signals includes:
- read
 - write
 - interrupt
 - clock
 - dictate the flow of operations.

for example:

A 'read' signal on the control bus orders a component to send data, while a 'write' signal prompts it to receive data.

advantages:

- ensures that all components operates in harmony, preventing data collisions or misleads.
- You can expand and adapt it based on the system's need, accommodating new features or configurations.
- Streamless operations and reduces potential bottle necks by managing control ~~at~~ signal separately.

disadvantages:

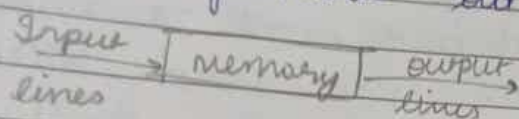
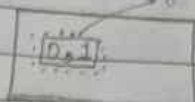
- The control bus needs to handle more signals as system grows, complicating design and troubleshooting.
- there's a limit to how many lines you can add to control bus, which might restrict the number of

control signals in highly advanced systems.

Read and Write operations in memory:-

A memory unit stores binary information in groups of bits called words.

Data input lines provides the information to be stored into the memory. Data output lines carry the information out from the memory.



The control lines read and write specifies the direction of transfer of data.

Basically, in the memory organization, there are 2^n memory locations indexing from 0 to $2^n - 1$. where, 'n' = address Buses.

We can describe the memory in terms of the bytes using the Formula:

$$N = 2^n \text{ bytes}$$

n = total address Buses.

N = Memory in Bytes.

For example:

$$1KB = 2^{10} \text{ Bytes}$$

$$64KB = 2^6 \times 2^{10} \text{ Bytes}$$

$$= 2^{16} \text{ Bytes}$$

$$4GB = 2^2 \times 2^{10} \text{ KB} \times 2^{10} \text{ (MB)} \times 2^{10} \text{ (GB)}$$

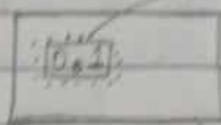
$$= 2^{32} \text{ Bytes}$$

control signals in highly advanced systems.

Read and Write operations in Memory:-

A memory unit stores binary information in groups of bits called words.

Data input lines provides the information to be stored into the memory. Data output lines carry the information out from the memory.



Input lines → Memory → Output lines

The control lines read and write specifies the direction of transfer of data.

Basically, in the memory organization, there are 2^n memory locations indexing from 0 to $2^n - 1$. Where, 'n' = address Buses.

We can describe the memory in terms of the bytes using the Formula:

$$N = 2^n \text{ bytes}$$

n = total address Buses.

N = Memory in Bytes.

For example:

$$1KB = 2^{10} \text{ Bytes}$$

$$64KB = 2^6 \times 2^{10} \text{ Bytes}$$

$$= 2^{16} \text{ Bytes}$$

$$4GB = 2^2 \times 2^{10} KB \times 2^{10} (MB) \times 2^{10} (GB)$$

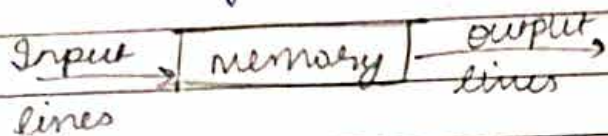
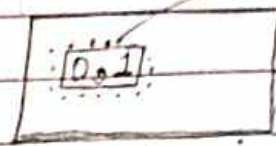
$$= 2^{32} \text{ Bytes}$$

control signals in highly advanced systems.

Read and Write operations in Memory:-

A memory unit stores binary information in groups of bits called words.

Data input lines provides the information to be stored into the memory, Data output lines carry the information out from the memory.



The control lines ~~read~~ read and write specifies the direction of transfer of data.

Basically, in the memory organization, there are 2^n memory locations indexing from 0 to $2^n - 1$.
Where, 'n' = address Buses.

We can describe the memory in terms of the bytes using the Formula:

$$N = 2^n \text{ bytes}$$

n = total address Buses.

N = Memory in Bytes.

For example:

$$1KB = 2^{10} \text{ Bytes}$$

$$64KB = 2^6 \times 2^{10} \text{ Bytes}$$

$$= 2^{16} \text{ Bytes}$$

$$4GB = 2^2 \times 2^{10} \text{ KB} \times 2^{10} \text{ (MB)} \times 2^{10} \text{ (GB)}$$

$$= 2^{32} \text{ Bytes}$$

Memory Address Register (MAR):

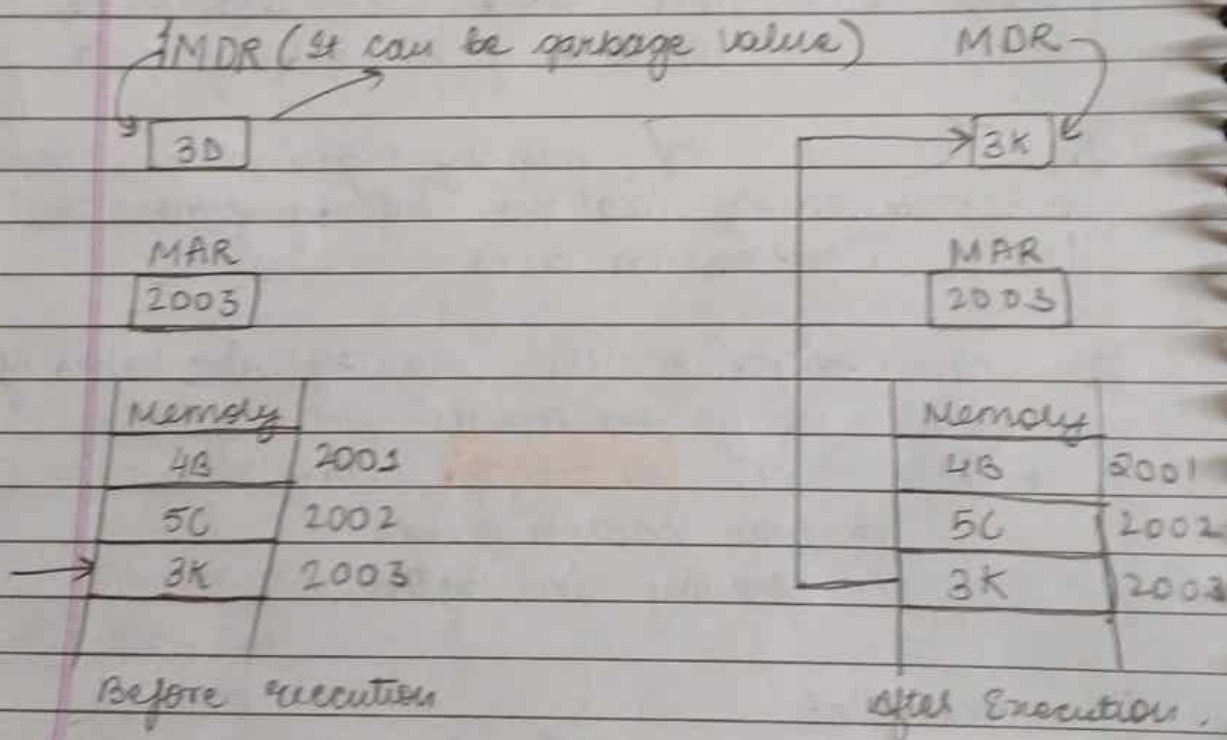
It is the Address Register used to store Address of the memory location where the operation is being performed.

Memory Data Register (MDR):

It is the data Register which is used to store the data on which the operation is being performed.

1. Memory Read Operation:

It transfers the desired word to address lines and activates the read control lines.



Memory Read Operation

In the above diagram, initially MDR can contain a garbage value and MAR is containing 2003 memory address.

Memory Address Register (MAR):

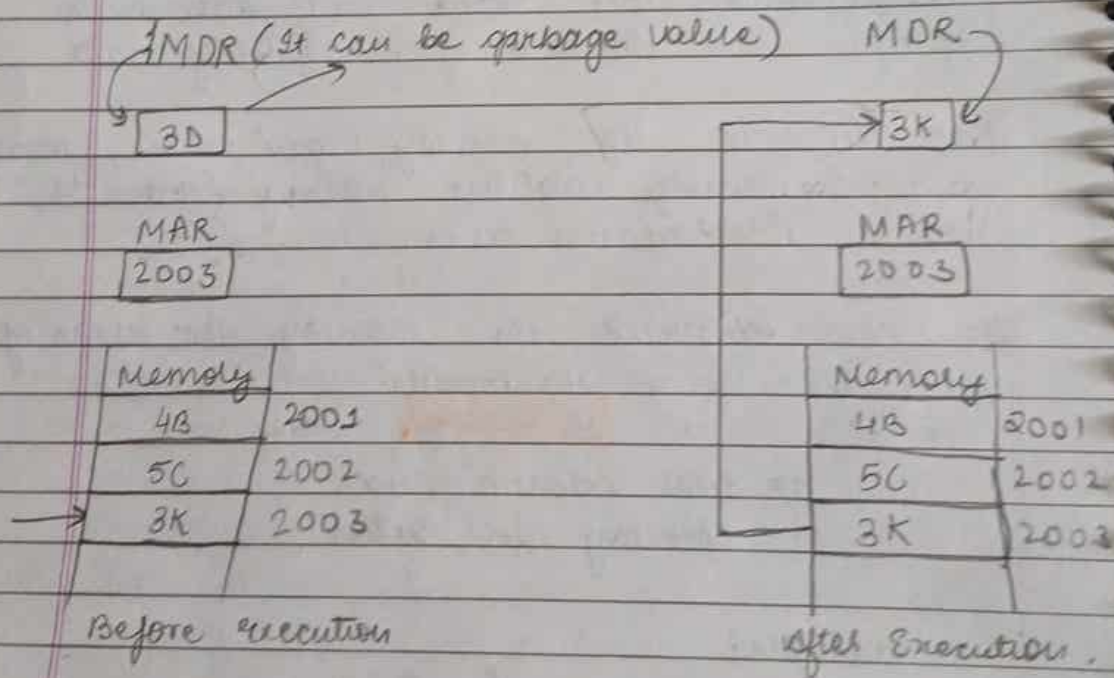
It is the address Register used to store address of the memory location where the operation is being performed.

Memory Data Register (MDR):

It is the data Register which is used to store the data on which the operation is being performed.

1. Memory Read Operation:

It transfers the desired word to address lines and activates the read control lines.



Memory Read Operation.

In the above diagram, initially MDR can contain a garbage value and MAR is containing 2003 memory address.

Memory Address Register (MAR):

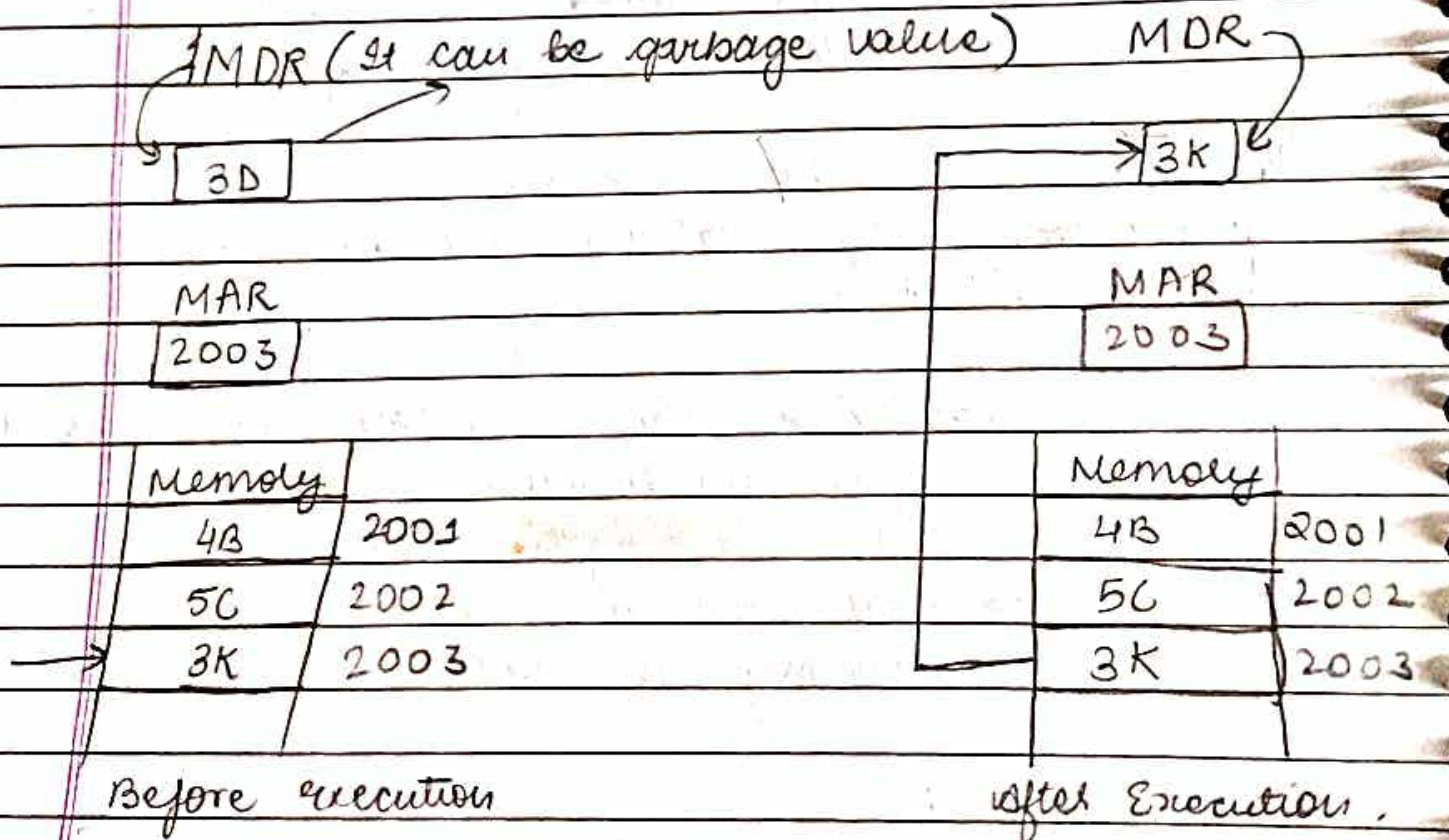
It is the address register used to store address of the memory location where the operation is being performed.

Memory Data Register (MDR):

It is the data register which is used to store the data on which the operation is being performed.

1. Memory Read Operation:

It transfers the desired word to address lines and activates the read control lines.



Memory Read operation.

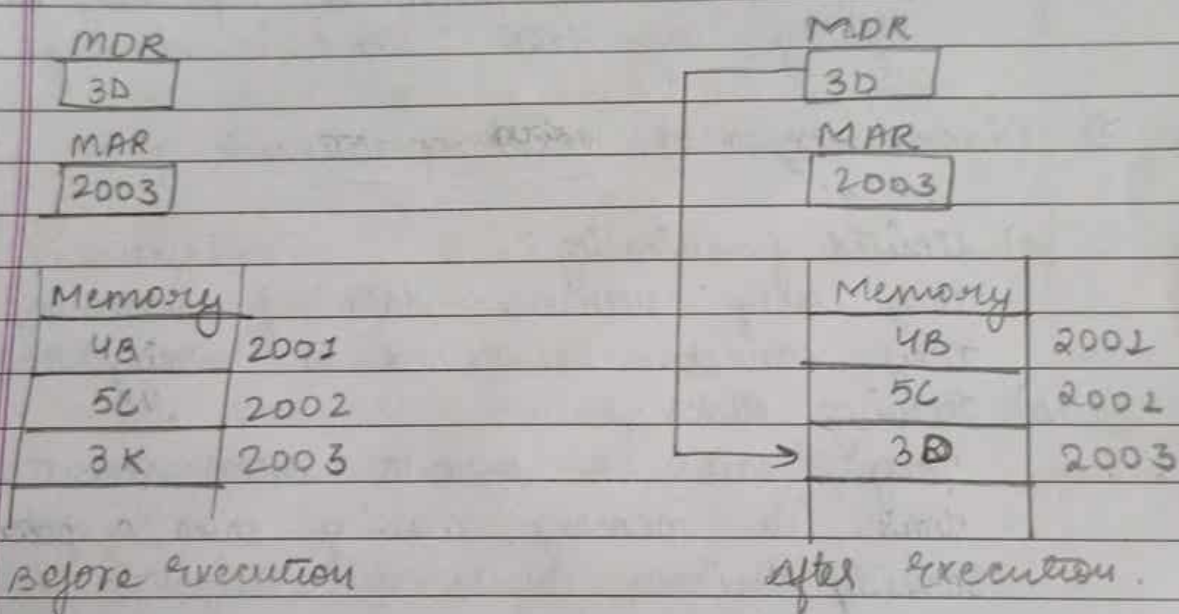
In the above diagram, initially MDR can contain a garbage value and MAR is containing 2003 memory address.

After the execution of read instruction, the data of memory location 2003 will be read and the MDR will get updated by the value of the 2003 memory location (3D).

2. Memory Write Operation:-

It transfers the address of the desired word to the address lines, transfers the data bits to be stored in memory of the data input lines.

Then it activates the write control line.

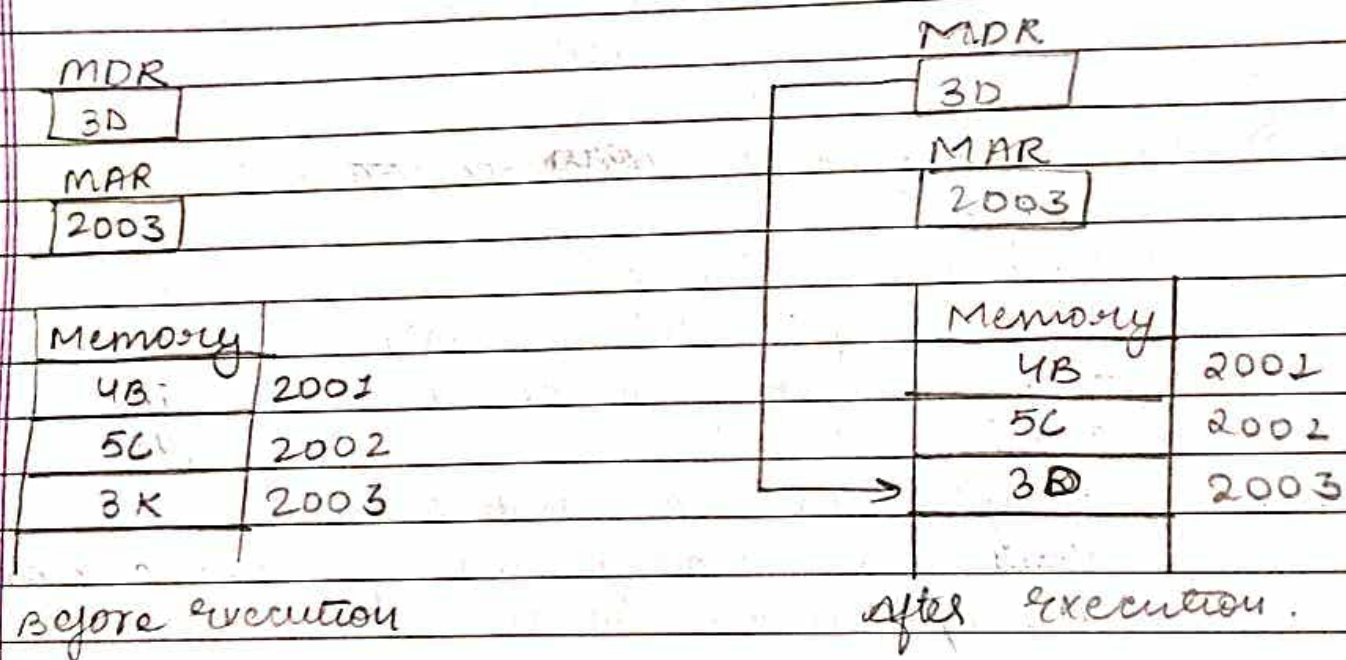


The MAR contains 2003 and MDR contains 3D. After the execution of write instructions 3D will be written at 2003 memory location.

After the execution of read instruction, the data of memory location 2003 will be read and the MDR will get updated by the value of the 2003 memory location (3D).

2. Memory Write Operation:-

It transfers the address of the desired word to the address lines, transfers the data bits to be stored in memory of the data input lines. Then it activates the write control line.



The MAR contains 2003 and MDR contains 3D. After the execution of write instructions 3D will be written at 2003 memory location.

>> Advantages of Read operation:-

a) Speed:

Read operations are generally faster than write operations since the data is already stored in the memory.

b) Efficiency:

They are more efficient since they do not require ~~the~~ ~~write~~ modifying the data in memory.

c) Non-destructive:

They can be performed repeatedly without affecting the data stored in memory.

>> Disadvantages of Read operations:

a) Limited functionality:

They only retrieve data from memory, so they cannot be used to modify data.

b) Security Risks:

They can be used to access sensitive data stored in memory, making them a potential security risk.

>> Advantages of Write operations:

a) Flexibility:

Write operations allow data to be modified, making them useful for storing and updating information in memory.

b) Dynamic:

Write operations allow data to be changed in

real time, making them essential for many computing applications.

c) Customization:

Write operations allows users to customize and personalize their computing experience by modifying stored data.

>> Disadvantages of write operations:

a) Slower:

They are slower than read operations since data needs to be modified and then written back to memory.

b) Overwriting Risk:

Write operations can overwrite existing data in memory, leading to data loss or corruption.

c) Wear and Tear:

Repeated write operations can cause wear and tear on memory cells, leading to reduced reliability and lifespan.

I/O Interface (Interrupt and DMA Mode)

The method that is used to transfer information between internal storage and external I/O devices is known as I/O interface.

Mode of Transfer:

The binary information that is received from an external device is usually stored in the memory unit.

CU merely processes the information but the target and source is always the memory unit.

Date _____
Page _____

Data Transfer to and from ~~can~~ the peripheral devices can be done in of the following 3 modes.

- i) Programmed I/O
- ii) Interrupt - initiated I/O
- iii) Direct memory access (DMA).

i) Programmed I/O:

It is due to the result of the I/O instruction that are written in the computer program.

Each data transfer is initiated by an instruction in the program.

Usually the transfer is from the CPU registers and memory.

In this case it requires constant monitoring by the CPU of the peripheral devices.

Example:

In this case the I/O device does not have direct access to the memory unit.

A transfer from I/O device to memory requires the execution of several instructions by the CPU, including an input instruction to transfer the data from device to CPU, & store instruction to transfer the data from CPU to memory.

Note: In programmed I/O the CPU stays in the program loop until the I/O

Date _____
Page _____

Data transfer to and from can be the peripheral devices can be done in of the following 3 modes.

- i) Programmed I/O
- ii) Interrupt - initiated I/O
- iii) Direct memory access (DMA).

i) Programmed I/O:

It is due to the result of the I/O instruction that are written in the computer program.

Each data transfer is initiated by an instruction in the program.

Usually the transfer is from the CPU register and memory.

In this case it requires constant monitoring by the CPU of the peripheral devices.

Example:

In this case the I/O device does not have direct access to the memory unit.

A transfer from I/O device to memory requires the execution of several instructions by the CPU, including an input instruction to transfer the data from device to CPU, & store instruction to transfer the data from CPU to memory.

Note: In programmed I/O the CPU stays in the program loop until the I/O

unit indicates that it is ready for data transfer.

A time consuming ~~process~~ process, keeps CPU busy throughout.

To avoid this we have :-

ii) ~~Interrupt~~ Interrupt - Initiated I/O:

By using interrupt facility and special commands to inform the interface to issue an interrupt request signal whenever the data is available from any device.

In the meantime the CPU can proceed for any other program execution.

The interface meanwhile keeps monitoring the device.

→ The I/O transfer rate is limited by the speed with which the processor can test and service a device.

→ The processor is tied up in managing a I/O transfer; a number of instructions must be executed for each I/O transfer.

iii) Direct Memory access:

The data transfer b/w a fast storage media such as magnetic disk and memory unit is limited by the speed of the CPU.

Thus, we can allow the peripherals directly communicate with each other using the memory buses, removing the intervention of the CPU.

During DMA, the CPU is idle and it has no control over the memory buses.

1. Bus grant request time
2. Transfer the data entire block data at transfer rate of device because the device is usually slow than the speed of at which the data can be transferred to CPU.
3. Release the control of the bus back to CPU so, - total time taken to transfer N bytes = Bus grant request time + (N) * (Memory transfer rate) + Bus release control time.

Formula:-

$$\text{T.T to transfer N bytes} = \text{Bus grant req. time} + (N) * (\text{Memory transfer rate}) + \text{Bus release control time}$$

- Time

- Memory in Bytes

- Memory transfer Rate

4. Buffer the byte into the Buffer.
5. Informs the CPU that the device has 1 byte to transfer (i.e. bus grant request)
6. Transfer the byte (at system bus speed)

During DMA, the CPU is idle and it has no control over the memory buses.

1. Bus grant request time
2. Transfer the ~~data~~ entire block data at transfer rate of device because the device is usually slow than the speed of at which the data can be transferred to CPU.
3. Release the control of the bus back to CPU so, total time taken to transfer N bytes = Bus grant request time + (N) * (Memory transfer rate) + Bus release control time.

Formula :-

$$\text{T.T to Transfer N Bytes} = \text{Bus grant req. time} + (N) * (\text{Memory transfer rate}) + \text{Bus release control time}$$

- Time
- Memory in Bytes
- Memory Transfer Rate

4. Buffer the byte into the buffer.
5. Informs the CPU that the device has 1 byte to transfer (i.e. bus grant request)
6. Transfer the byte (at system bus speed)

Advantages:

a) standardization:

Different devices can be connected to a computer using the same interface, which makes it easier to swap out devices and reduce the need for specialized hardware.

b) Modularity:

With I/O interfaces different devices can be added/removed from a computer without affecting the other components, which makes it easier to upgrade/replace a faulty device without affecting the rest of the system.

c) Efficiency:

I/O ^{interfaces} can transfer the data b/w the computer & the external devices at high speeds, allowing for faster data transfer & processing times.

d) compatibility:

Users can ~~use~~ choose from a variety of devices that are compatible with their computer's I/O interface.

Disadvantages:

a) Cost: They can be expensive.

b) Complexity: This can be a disadvantage for users who are not familiar with the technical aspects of computer hardware.

c) Compatibility: There still can be compatibility issues with certain devices. In some cases, additional drivers may need to be installed.

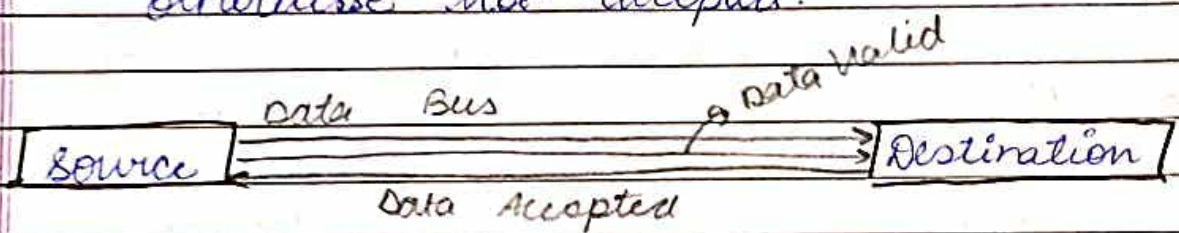
to ensure proper functionality,
d) Security risks: Hackers can exploit vulnerabilities in I/O interfaces to gain unauthorized access to a computer system/steal data.

>> Concept of Handshaking:

1) When source initiates the data transfer Source process.

Initiated it consists of signals:

Handshaking: → Data VALID: If ON tells the data on the Data Bus is valid otherwise invalid.
→ Data Accepted: If ON tells data is accepted otherwise not accepted.



i) source places data on the data bus & enables data valid signal.

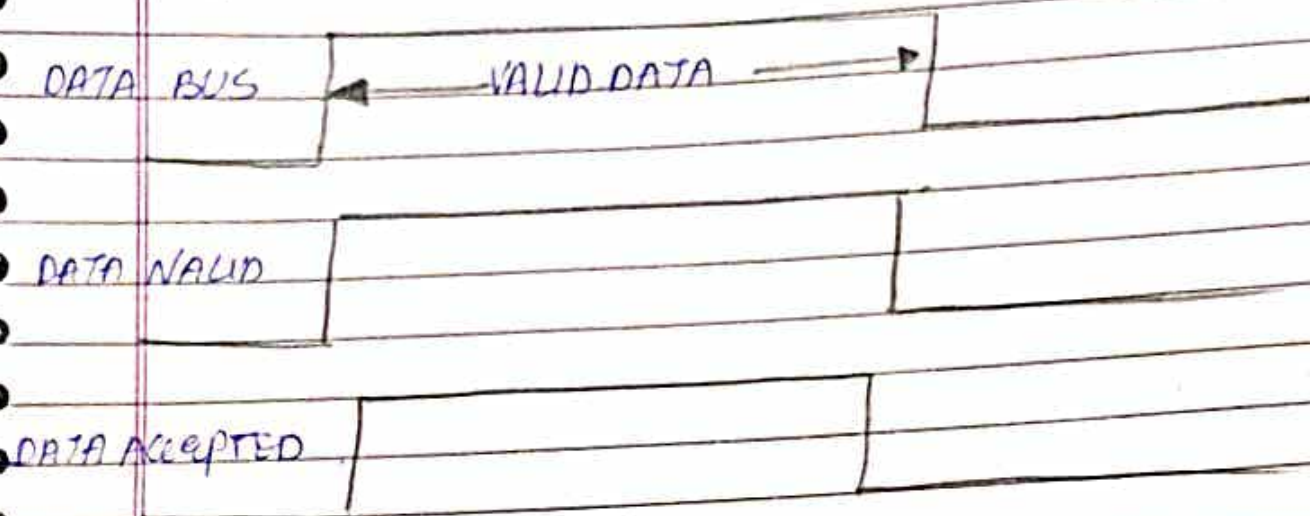
ii) Destination Accepts data from the data bus and enable data accepted signal.

iii) After this, disable Data valid signal means data on data bus is invalid now.

iv) Disable Data accepted signal & the process ends.

Now, there is surety that destination has read the data from the data bus through data accepted signal.

Signals can be seen as:-

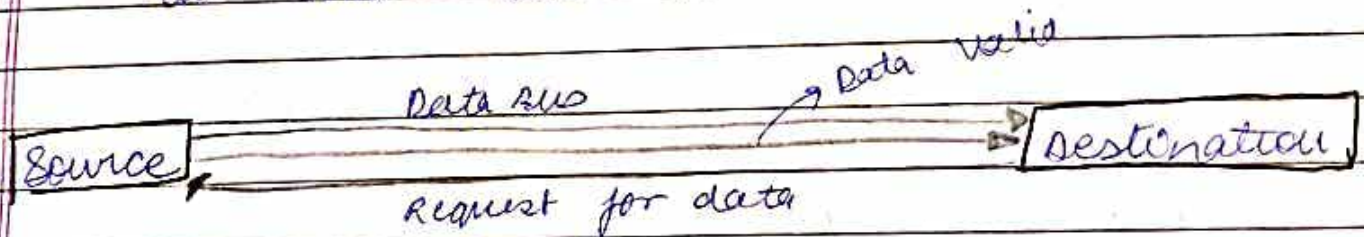


2). When destination initiates the process of destination data transfer.

Initiated it consists of signals:

Handshaking: → REQUEST FOR DATA: If DN requests for putting data on the data bus.

→ DATA VALID: If DN tells the data is valid on the data bus otherwise invalid data.



- i) When destination is ready to receive, data, request for data signal gets activated.
- ii) Source in response puts data on the data bus and enables data valid signal.
- iii) Destination then accepts the data on the data bus and after accepting the data, ~~data~~ disabled request for data signal.
- iv) At last, data valid signal gets disabled when data on the data bus is no more valid data.

Signals can be seen as $\Rightarrow$

Data Bus VALID DATA $\rightarrow$

data valid

data accepted

Features :

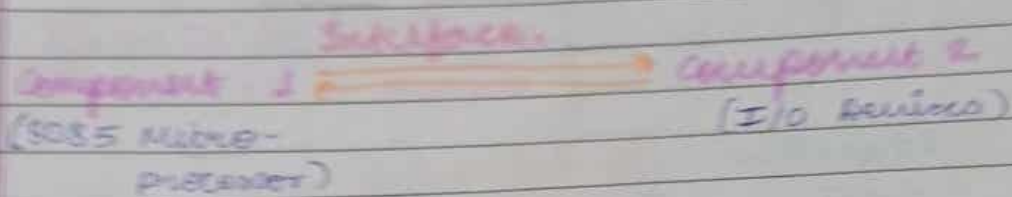
$\rightarrow$ Polling

A technique used to check the status of a device periodically to determine if it has completed a data transfer operation. With asynchronous input-output, the CPU can poll the device periodically to check if the data transfer has finished. If the transfer is complete, the CPU can then retrieve the data from the device.

Interfacing I/O Devices with 8085 Microprocessor:

Memory Mapped and Peripheral Mapped.

Interface is the path of communication b/w two components.



② Peripheral Mapped I/O Interfacing:
I/O devices are treated differently from memory chips.

Control signals $\rightarrow$ $\overline{IO/\overline{M}}$, $\overline{RD}$, $\overline{WR}$

$IO + \overline{RD} \rightarrow \overline{IOR}$ } Read/Write
 $IO + \overline{WR} \rightarrow \overline{IOW}$ }

0 - active
1 - inactive

	$\overline{IO/\overline{M}}$	$\overline{RD}$	$\overline{WR}$	$\overline{IOR}$	$\overline{IOW}$	
Active signals	1	0	1	0	1	I/O Read operation
	1	1	0	1	0	I/O Write operation
	0	X	X	1	1	Memory Read/Write operation

$\overline{IN}$ $\rightarrow$ used to access input devices. 64KB
 $\overline{OUT}$ $\rightarrow$ used to access output devices. Memory

I/O devices $\rightarrow$ 8-bit address. 00 to FF
 0 to 255

* Maximum of 256 (2⁸) input devices & 256 output devices can be interfaced with 8085.

Interfacing I/O Devices with 8085 Microprocessor:

g/o devices are connected to the microprocessor via Memory Mapped and Peripheral Mapped.

Interface is the path of communication b/w two components.

Interface.

Component 1
(8085 Micro-processor)

Component 2
(I/O Devices)

① Peripheral Mapped I/O Interfacing:

I/O devices are treated differently from memory chips

Control signals $\rightarrow$ $\overline{IO/\overline{M}}$, $\overline{RD}$, $\overline{WR}$

II

$IO + \overline{RD} \rightarrow \overline{IOR}$ } Read/write
 $IO + \overline{WR} \rightarrow \overline{IOW}$ }

0 - active
1 - inactive

	$\overline{IO/\overline{M}}$	$\overline{RD}$	$\overline{WR}$	$\overline{IOR}$	$\overline{IOW}$	
status of control signals $\rightarrow$	1	0	1	0	1	I/O Read operation.
	1	1	0	1	0	I/O write operation.
	0	X	X	1	1	Memory Read/Write operation.

IN $\rightarrow$ used to access input devices. 64KB

OUT $\rightarrow$ used to access output devices. Memory

unique
I/O devices $\rightarrow$ 8-bit address. 00 to FF
0 to 256

* maximum of 256 (28) input devices & 256 output devices can be interfaced with 8085.

Interfacing I/O Devices with 8085 Microprocessor:

^{I/O devices as memory chips}
Memory Mapped and Peripheral Mapped.

Interface is the path of communication b/w two components.

Interface:



① Peripheral Mapped I/O Interfacing:
I/O devices are treated differently from memory chips

Control signals → $\overline{IO/\overline{M}}$, $\overline{RD}$, $\overline{WR}$

$IO + \overline{RD} \rightarrow \overline{IOR}$ } Read/Write
 $IO + \overline{WR} \rightarrow \overline{IOW}$ }

0 - Active Low

1 - Inactive High

	$\overline{IO/\overline{M}}$	$\overline{RD}$	$\overline{WR}$	$\overline{IOR}$	$\overline{IOW}$	
Active Low	1	0	1	0	1	I/O Read operation
Active Low	1	1	0	1	0	I/O Write operation
Active Low	0	X	X	1	1	Memory read, write operation

IN → used to access input devices. 64KB

OUT → used to access output devices. Memory

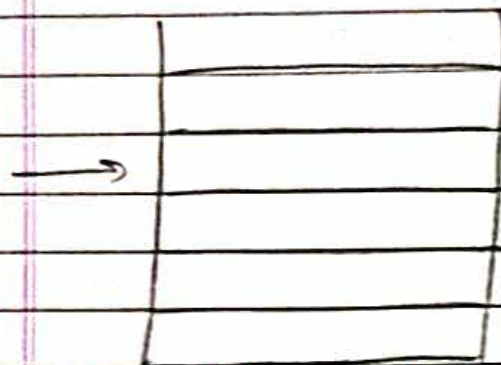
I/O devices → ^{unique} 8-bit address. 00 to FF

0 to 256

* maximum of 256 (28) input devices & 256 output devices can be interfaced with 8085.

② Memory Mapped I/O Interfacing:

→ Each input/output devices is treated like as if it is a memory location.



16-bit Address.

Control signals:

$\overline{MEMR} \rightarrow \overline{IO/\overline{M}} + \overline{RD}$

$\overline{MEMW} \rightarrow \overline{IO/\overline{M}} + \overline{RW}$

1 Inactive

0 Active.

$\overline{IO/\overline{M}}$	$\overline{RD}$	$\overline{WR}$	$\overline{MEMR}$	$\overline{MEMW}$
0	0	1	0	1
0	1	0	1	0

Memory Related instructions are used:

like: LDA 2000H
LDAX B
MOV A, M

Memory size

8085 = 64KB

Micro processor

Memory I/O Devices

Memory mapped

1. • 16 bit address
2. • $\overline{MEMW}$ and $\overline{MEMR}$: Control signals.
3. • Memory related instructions are used.
4. • Data transfer takes place any memory register and I/O.
5. • 3 bytes instruction.
6. • Uses more space.
7. • 64KB of memory is shared b/w I/O devices & the memory.
8. • $65536 = 64KB$
9. • More hardware is required.
10. • ALU operations perform directly on data.

Peripheral devices

- 8-bit address
- Control signal: $\overline{IOB}$ and $\overline{IOW}$
- IN and OUT instructions are used.
- Data transfer takes place accumulators and I/O devices.
- 2 byte instructions
- Uses less space
- No sharing of memory is done.
- Max. no. of I/O devices 256.
- Less hardware is required.
- ALU operations perform indirectly on data.